

Отчёт по лабораторной работе №3

Компьютерный практикум по статистическому анализу

Надежда Александровна Рогожина

Содержание

1	Цель работы	6
2	Задание	7
3	Теоретическое введение	8
4	Выполнение лабораторной работы	9
5	Выводы	31
	Список литературы	32

Список иллюстраций

4.1	Циклы	9
4.2	Циклы	10
4.3	Циклы	10
4.4	Циклы	10
4.5	Циклы	11
4.6	Условные выражения	11
4.7	Функции	11
4.8	Функции	12
4.9	Функции	12
4.10	Функции	13
4.11	Функции	13
4.12	Функции	14
4.13	Пакеты	14
4.14	Целые числа	15
4.15	Целые числа	15
4.16	squares	16
4.17	squares	17
4.18	squares_arr	18
4.19	squares_arr	19
4.20	Четное/нечетное	20
4.21	add_one	20
4.22	broadcast()	21
4.23	Задание 5	21
4.24	Задание 5	22
4.25	Задание 6	22
4.26	Матрицы Z и E	23
4.27	Задание 7	24
4.28	Задание 7	25
4.29	outer	26
4.30	Задание 8	26
4.31	Задание 8	27
4.32	Задание 8	27
4.33	Задание 9	28
4.34	Задание 9	28
4.35	Задание 9	29
4.36	Задание 9	29

4.37	Задание 9	30
------	---------------------	----

Список таблиц

1 Цель работы

Основная цель работы — освоить применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 3.2.
2. Выполните задания для самостоятельной работы (раздел 3.4).

3 Теоретическое введение

Научные вычисления традиционно требуют высочайшей производительности, однако эксперты по доменам в значительной степени перенесены на более медленные динамические языки для ежедневной работы. К счастью, методы современного языка и компилятора позволяют в основном устранить компромисс производительности и обеспечивать достаточно продуктивную среду для прототипирования и достаточно эффективно для развертывания применений, интенсивных производительности. Язык программирования Julia заполняет эту роль: это гибкий динамический язык, подходящий для научных и численных вычислений, с производительностью, сравнимой с традиционными статичными языками.

4 Выполнение лабораторной работы

Первым заданием было повторить примеры, где были показаны основы работы с циклами (рис. 4.1, рис. 4.2, рис. 4.3, рис. 4.4, рис. 4.5), условными выражениями (рис. 4.6), функциями (рис. 4.7, рис. 4.8, рис. 4.9, рис. 4.10, рис. 4.11, рис. 4.12) и пакетами (рис. 4.13).

Циклы while и for

```
In [4]: n = 0
        while n < 10
            n += 1
            println(n)
        end
1
2
3
4
5
6
7
8
9
10

In [6]: myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
        i = 1
        while i <= length(myfriends)
            friend = myfriends[i]
            println("Hi $friend, it's great to see you!")
            i += 1
        end
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
```

Рисунок 4.1: Циклы

```
In [10]: for n in 1:2:10
        println(n)
        end

1
3
5
7
9

In [11]: myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]

        for friend in myfriends
            println("Hi $friend, it's great to see you!")
        end

Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
```

Рисунок 4.2: Циклы

```
In [16]: m, n = 5, 5
        A = fill{0, (m, n)}
        for i in 1:m
            for j in 1:n
                A[i, j] = i + j
            end
        end
        A

Out[16]: 5×5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10
```

Рисунок 4.3: Циклы

```
In [14]: B = fill{0, (m, n)}
        for i in 1:m, j in 1:n
            B[i, j] = i + j
        end
        B

Out[14]: 5×5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10
```

Рисунок 4.4: Циклы

```
In [15]: C = [i + j for i in 1:m, j in 1:n]
C
```

```
Out[15]: 5×5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10
```

Рисунок 4.5: Циклы

Условные выражения

```
In [30]: n = [3,5,15]
for N in n
    if (N % 3 == 0) && (N % 5 == 0)
        println("FizzBuzz")
    elseif N % 3 == 0
        println("Fizz")
    elseif N % 5 == 0
        println("Buzz")
    else
        println(N)
    end
end
Fizz
Buzz
FizzBuzz
```

```
In [32]: x = 5
y = 10
(x>y) ? x : y
```

```
Out[32]: 10
```

Рисунок 4.6: Условные выражения

Функции

```
In [34]: function sayhi(name)
println("Hi $name, it's great to see you!")
end
```

```
Out[34]: sayhi (generic function with 1 method)
```

```
In [35]: function f(x)
x^2
end
```

```
Out[35]: f (generic function with 1 method)
```

```
In [36]: sayhi("C-3PO")
f(42)
```

```
Hi C-3PO, it's great to see you!
```

```
Out[36]: 1764
```

Рисунок 4.7: Функции

```
In [37]: sayhi2(name) = println("Hi $name, it's great to see you!")
        f2(x) = x^2
Out[37]: f2 (generic function with 1 method)

In [38]: sayhi("C-3PO")
        f(42)
        Hi C-3PO, it's great to see you!
Out[38]: 1764
```

Рисунок 4.8: Функции

```
In [43]: v = [3, 5, 2]
        sort(v)
Out[43]: 3-element Vector{Int64}:
         2
         3
         5

In [44]: v
Out[44]: 3-element Vector{Int64}:
         3
         5
         2

In [45]: sort!(v)
Out[45]: 3-element Vector{Int64}:
         2
         3
         5

In [46]: v
Out[46]: 3-element Vector{Int64}:
         2
         3
         5
```

Рисунок 4.9: Функции

```
In [47]: map(f, [1, 2, 3])
Out[47]: 3-element Vector{Int64}:
 1
 4
 9
```

```
In [49]: map(x -> x^3, [1, 2, 3])
Out[49]: 3-element Vector{Int64}:
 1
 8
27
```

```
In [50]: broadcast(f, [1, 2, 3])
Out[50]: 3-element Vector{Int64}:
 1
 4
 9
```

Рисунок 4.10: Функции

```
In [51]: A = [i + 3*j for j in 0:2, i in 1:3]
Out[51]: 3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8  9
```

```
In [52]: f(A)
Out[52]: 3×3 Matrix{Int64}:
30  36  42
66  81  96
102 126 150
```

```
In [53]: B = f.(A)
Out[53]: 3×3 Matrix{Int64}:
 1  4  9
16 25 36
49 64 81
```

Рисунок 4.11: Функции

```

In [54]: A .+ 2 .* f.(A) ./ A
Out[54]: 3×3 Matrix{Float64}:
 3.0  6.0  9.0
12.0 15.0 18.0
21.0 24.0 27.0

In [55]: @. A + 2 * f(A) / A
Out[55]: 3×3 Matrix{Float64}:
 3.0  6.0  9.0
12.0 15.0 18.0
21.0 24.0 27.0

In [56]: broadcast(x -> x + 2 * f(x) / x, A)
Out[56]: 3×3 Matrix{Float64}:
 3.0  6.0  9.0
12.0 15.0 18.0
21.0 24.0 27.0

In [61]: 2 + (2*4) / 2
Out[61]: 6.0

```

Рисунок 4.12: Функции

```

Пакеты
In [63]: Pkg.add("Colors")
using Colors
palette = distinguishable_colors(100)

Resolving package versions...
Installed Colors - v0.13.1
Updating 'C:\Users\Igor\Julia\environments\v1.11\Project.toml'
[5ae59095] + Colors v0.13.1
Updating 'C:\Users\Igor\Julia\environments\v1.11\Manifest.toml'
[5ae59095] + Colors v0.13.0 => v0.13.1
Precompiling project...
 884.4 ms / Colors
3385.9 ms / SparseMatrixColorings + SparseMatrixColoringsColorExt
6889.1 ms / ColorSchemes
19462.0 ms / Plots.jl
5881.0 ms / PlotsThemes
9083.5 ms / RecipesPipeline
15550.0 ms / Plots
14356.0 ms / Plots + UnitfulExt
14079.3 ms / Plots + Themed
9 dependencies successfully precompiled in 218 seconds. 471 already precompiled.

Out[63]:


In [64]: rand(palette, 3, 3)
Out[64]:


```

Рисунок 4.13: Пакеты

Далее, необходимо было выполнить *Задания для самостоятельного выполнения*:

1. Используя циклы `while` и `for`: – выведите на экран целые числа от 1 до 100 и напечатайте их квадраты (рис. 4.14, рис. 4.15); – создайте словарь `squares`, который будет содержать целые числа в качестве ключей и квадраты в качестве их пар-значений (рис. 4.16, рис. 4.17); – создайте массив `squares_arr`, содержащий квадраты всех чисел от 1 до 100 (рис. 4.18, рис. 4.19).

Задания для самостоятельного выполнения

```
In [58]: i = 1
while i <= 100
    println("Число: $i, квадрат(i): $(i^2)")
    i += 1
end
```

Число: 82, квадрат(i): 6724
Число: 83, квадрат(i): 6889
Число: 84, квадрат(i): 7056
Число: 85, квадрат(i): 7225
Число: 86, квадрат(i): 7396
Число: 87, квадрат(i): 7569
Число: 88, квадрат(i): 7744
Число: 89, квадрат(i): 7921
Число: 90, квадрат(i): 8100
Число: 91, квадрат(i): 8281
Число: 92, квадрат(i): 8464
Число: 93, квадрат(i): 8649
Число: 94, квадрат(i): 8836
Число: 95, квадрат(i): 9025
Число: 96, квадрат(i): 9216
Число: 97, квадрат(i): 9409
Число: 98, квадрат(i): 9604
Число: 99, квадрат(i): 9801
Число: 100, квадрат(i): 10000

Рисунок 4.14: Целые числа

```
In [69]: for i in 1:1:100
    println("Число: $i, квадрат(i): $(i^2)")
end
```

Число: 82, квадрат(i): 6724
Число: 83, квадрат(i): 6889
Число: 84, квадрат(i): 7056
Число: 85, квадрат(i): 7225
Число: 86, квадрат(i): 7396
Число: 87, квадрат(i): 7569
Число: 88, квадрат(i): 7744
Число: 89, квадрат(i): 7921
Число: 90, квадрат(i): 8100
Число: 91, квадрат(i): 8281
Число: 92, квадрат(i): 8464
Число: 93, квадрат(i): 8649
Число: 94, квадрат(i): 8836
Число: 95, квадрат(i): 9025
Число: 96, квадрат(i): 9216
Число: 97, квадрат(i): 9409
Число: 98, квадрат(i): 9604
Число: 99, квадрат(i): 9801
Число: 100, квадрат(i): 10000

Рисунок 4.15: Целые числа

```
In [75]: squares = Dict()
         i = 1
         while i <= 100
             squares[i] = i^2
             i += 1
         end
         squares

Out[75]: Dict{Any, Any} with 100 entries:
  5 => 25
 56 => 3136
 35 => 1225
 55 => 3025
 60 => 3600
 30 => 900
 32 => 1024
 6 => 36
 67 => 4489
 45 => 2025
 73 => 5329
 64 => 4096
 90 => 8100
 4 => 16
 13 => 169
 54 => 2916
 63 => 3969
 86 => 7396
 91 => 8281
 62 => 3844
 58 => 3364
 52 => 2704
 12 => 144
 28 => 784
 75 => 5625
 ⋮ => ⋮

In [76]: squares[45]

Out[76]: 2025
```

Рисунок 4.16: squares

```
In [70]: squares = Dict()
         for i in 1:1:100
           squares[i] = i^2
         end
         squares
```

Out[70]: Dict{Any, Any} with 100 entries

5	=>	25
56	=>	3136
35	=>	1225
55	=>	3025
60	=>	3600
30	=>	900
32	=>	1024
6	=>	36
67	=>	4489
45	=>	2025
73	=>	5329
64	=>	4096
90	=>	8100
4	=>	16
13	=>	169
54	=>	2916
63	=>	3969
86	=>	7396
91	=>	8281
62	=>	3844
58	=>	3364
52	=>	2704
12	=>	144
28	=>	784
75	=>	5625
⋮	=>	⋮

```
In [73]: squares[45]
```

Out[73]: 2025

Рисунок 4.17: squares

```
In [77]: squares_arr = [i^2 for i in 1:1:100]
```

```
Out[77]: 100-element Vector{Int64}:
```

```
 1  
 4  
 9  
16  
25  
36  
49  
64  
81  
100  
121  
144  
169  
⋮  
7921  
8100  
8281  
8464  
8649  
8836  
9025  
9216  
9409  
9604  
9801  
10000
```

Рисунок 4.18: squares_arr

```
In [80]: squares_arr = ones(100)
i = 1
while i <= 100
    squares_arr[i] = i^2
    i += 1
end
squares_arr

Out[80]: 100-element Vector{Float64}:
 1.0
 4.0
 9.0
16.0
25.0
36.0
49.0
64.0
81.0
100.0
121.0
144.0
169.0
 ⋮
7921.0
8100.0
8281.0
8464.0
8649.0
8836.0
9025.0
9216.0
9409.0
9604.0
9801.0
10000.0
```

Рисунок 4.19: squares_arr

2. Напишите условный оператор, который печатает число, если число чётное, и строку «нечётное», если число нечётное. Перепишите код, используя тернарный оператор (рис. 4.20).

```

In [86]: numbers = [3, 4, 0]
for num in numbers
    if num % 2 == 0
        println(num)
    elseif num % 2 != 0
        println("нечётное")
    else
        println("число = 0")
    end
end

нечётное
4
0

In [92]: num = 4
println((num % 2 == 0) ? num : "нечётное")
num = 3
println((num % 2 == 0) ? num : "нечётное")

4
нечётное

```

Рисунок 4.20: Четное/нечетное

3. Напишите функцию `add_one`, которая добавляет 1 к своему входу (рис. 4.21).

```

In [93]: function add_one(x)
           x += 1
       end

Out[93]: add_one (generic function with 1 method)

In [94]: add_one(5)

Out[94]: 6

```

Рисунок 4.21: `add_one`

4. Используйте `map()` или `broadcast()` для задания матрицы $\mathbb{N}$, каждый элемент которой увеличивается на единицу по сравнению с предыдущим (рис. 4.22).

```

In [115]: function create_matrix(row, col)
           rows = row
           cols = col
           A = fill(0, (rows, cols))
           k = 1
           for i in 1:rows, j in 1:cols
               A[i,j] = k
               k += 1
           end
           A
       end

Out[115]: create_matrix (generic function with 1 method)

In [118]: create_matrix(3,4)

Out[118]: 3x4 Matrix{Int64}:
 1  2  3  4
 5  6  7  8
 9 10 11 12

```

Рисунок 4.22: broadcast()

5. Задайте матрицу A (рис. 4.23).

1. Найдите A^3 (рис. 4.23)
2. Замените третий столбец матрицы A на сумму второго и третьего столбцов (рис. 4.24)

```

A = [[1,5,-2] [1, 2, -1] [3, 6, -3]]

3x3 Matrix{Int64}:
 1  1  3
 5  2  6
-2 -1 -3

function f3(x)
    x^3
end
f3(A)

3x3 Matrix{Int64}:
 0  0  0
 0  0  0
 0  0  0

```

Рисунок 4.23: Задание 5

```
In [135]: for i in 1:3
           A[i,3] = A[i,1] + A[i,2]
         end
```

```
In [136]: A
```

```
Out[136]: 3×3 Matrix{Int64}:
           1   1   2
           5   2   7
          -2  -1  -3
```

Рисунок 4.24: Задание 5

6. Создайте матрицу B с элементами $B_{i1} = 10, B_{i2} = -10, B_{i3} = 10, i = 1, 2, \dots, 15$. Вычислите матрицу $C = B^T B$ (рис. 4.25)

```
In [138]: rows = 15
           cols = 3
           B = fill{0, (rows, cols)}
           for i in 1:15
               B[i, 1] = 10
               B[i, 2] = -10
               B[i, 3] = 10
           end
           B
```

```
Out[138]: 15×3 Matrix{Int64}:
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
           10  -10  10
```

```
In [141]: B'B
```

```
Out[141]: 3×3 Matrix{Int64}:
          1500  -1500  1500
          -1500  1500  -1500
           1500  -1500  1500
```

Рисунок 4.25: Задание 6

7. Создайте матрицу Z размерности 6×6 , все элементы которой равны нулю, и матрицу E , все элементы которой равны 1 (рис. 4.26). Используя цикл `while`

или `for` и закономерности расположения элементов, создайте следующие матрицы размерности 6×6 (рис. 4.27, рис. 4.28).

```
In [143]: Z = fill(0, (6,6))
Out[143]: 6×6 Matrix{Int64}:
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0

In [144]: E = ones(6,6)
Out[144]: 6×6 Matrix{Float64}:
 1.0  1.0  1.0  1.0  1.0  1.0
 1.0  1.0  1.0  1.0  1.0  1.0
 1.0  1.0  1.0  1.0  1.0  1.0
 1.0  1.0  1.0  1.0  1.0  1.0
 1.0  1.0  1.0  1.0  1.0  1.0
 1.0  1.0  1.0  1.0  1.0  1.0
```

Рисунок 4.26: Матрицы Z и E

```
In [151]: Z1 = fill(0,(6,6))
for i in 1:6, j in 1:6
    if abs(i-j) == 1
        Z1[i,j] = E[i,j]
    end
end
Z1
```

```
Out[151]: 6×6 Matrix{Int64}:
 0  1  0  0  0  0
 1  0  1  0  0  0
 0  1  0  1  0  0
 0  0  1  0  1  0
 0  0  0  1  0  1
 0  0  0  0  1  0
```

```
In [179]: Z2 = fill(0,(6,6))
for i in 1:6, j in 1:6
    if (i==j) || abs(i-j) == 2
        Z2[i,j] = E[i,j]
    end
end
Z2
```

```
Out[179]: 6×6 Matrix{Int64}:
 1  0  1  0  0  0
 0  1  0  1  0  0
 1  0  1  0  1  0
 0  1  0  1  0  1
 0  0  1  0  1  0
 0  0  0  1  0  1
```

Рисунок 4.27: Задание 7


```

In [209]: Z3 = fill(0,(6,6))
for i in 1:6, j in 1:6
    if (i + j) == 5 || (i+j) == 7 || (i+j) == 9
        Z3[i,j] = E[i,j]
    end
end
Z3

Out[209]: 6×6 Matrix{Int64}:
 0  0  0  1  0  1
 0  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  0
 1  0  1  0  0  0

In [183]: Z4 = fill(0,(6,6))
for i in 1:6, j in 1:6
    if abs(i-j) % 2 == 0
        Z4[i,j] = E[i,j]
    end
end
Z4

Out[183]: 6×6 Matrix{Int64}:
 1  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  1

```

Рисунок 4.28: Задание 7

8. В языке R есть функция `outer()`. Фактически, это матричное умножение с возможностью изменить применяемую операцию (например, заменить произведение на сложение или возведение в степень).

1. Напишите свою функцию, аналогичную функции `outer()` языка R. Функция должна иметь следующий интерфейс: `outer(x,y,operation)`. Таким образом, функция вида `outer(A, B, *)` должна быть эквивалентна произведению матриц $\mathbb{R}^n \times \mathbb{R}^m$ и $\mathbb{R}^m \times \mathbb{R}^k$ соответственно (рис. 4.29).
2. Используя написанную вами функцию `outer()`, создайте матрицы следующей структуры (рис. 4.30, рис. 4.31, рис. 4.32).

```
In [230]: function outer(x, y, operation)
           if operation == "+"
               x = x + y;
           elseif operation == "-"
               x = x - y;
           elseif operation == "*"
               x = x * y;
           elseif operation == "^"
               x = x ^ y;
           elseif operation == "%"
               x = x % y;
           end
       end

Out[230]: outer (generic function with 1 method)
```

Рисунок 4.29: outer

```
In [220]: x = collect(0:4)
           y = collect(0:4)
           outer(x,y,"+")

Out[220]: 5×5 Matrix{Int64}:
           0  1  2  3  4
           1  2  3  4  5
           2  3  4  5  6
           3  4  5  6  7
           4  5  6  7  8

In [228]: x = collect(0:4)
           y = collect(1:5)
           outer(x,y,"^")

Out[228]: 5×5 Matrix{Int64}:
           0  0  0  0  0
           1  1  1  1  1
           2  4  8  16  32
           3  9  27  81  243
           4  16  64  256  1024
```

Рисунок 4.30: Задание 8

```
In [231]: x = collect(0:4)
y = collect(0:4)
outer(outer(x,y,"+"), 5, "%")
```

```
Out[231]: 5×5 Matrix{Int64}:
 0  1  2  3  4
 1  2  3  4  0
 2  3  4  0  1
 3  4  0  1  2
 4  0  1  2  3
```

```
In [267]: x = collect(0:9)
y = collect(0:9)
outer(outer(x,y,"+"), 10, "%")
```

```
Out[267]: 10×10 Matrix{Int64}:
 0  1  2  3  4  5  6  7  8  9
 1  2  3  4  5  6  7  8  9  0
 2  3  4  5  6  7  8  9  0  1
 3  4  5  6  7  8  9  0  1  2
 4  5  6  7  8  9  0  1  2  3
 5  6  7  8  9  0  1  2  3  4
 6  7  8  9  0  1  2  3  4  5
 7  8  9  0  1  2  3  4  5  6
 8  9  0  1  2  3  4  5  6  7
 9  0  1  2  3  4  5  6  7  8
```

Рисунок 4.31: Задание 8

```
In [266]: x = collect(0:8)
y = [0,8,7,6,5,4,3,2,1]
outer(outer(x, y, "+"), 9, "%")
```

```
Out[266]: 9×9 Matrix{Int64}:
 0  8  7  6  5  4  3  2  1
 1  0  8  7  6  5  4  3  2
 2  1  0  8  7  6  5  4  3
 3  2  1  0  8  7  6  5  4
 4  3  2  1  0  8  7  6  5
 5  4  3  2  1  0  8  7  6
 6  5  4  3  2  1  0  8  7
 7  6  5  4  3  2  1  0  8
 8  7  6  5  4  3  2  1  0
```

Рисунок 4.32: Задание 8

9. Решите следующую систему линейных уравнений с 5 неизвестными (рис. 4.33).

```

In [281]: A = [[1, 2, 3, 4, 5] [2, 1, 2, 3, 4] [3, 2, 1, 2, 3] [4, 3, 2, 1, 2] [5, 4, 3, 2, 1]]
          y = [7, -1, -3, 5, 17]
          x = A \ y
Out[281]: 5-element Vector{Float64}:
 -1.9999999999999987
  2.9999999999999996
  4.999999999999998
  2.0000000000000001
 -4.0

```

Рисунок 4.33: Задание 9

10. Создайте матрицу M размерности 6×10 , элементами которой являются целые числа, выбранные случайным образом с повторениями из совокупности $1, 2, \dots, 10$ (рис. 4.33).

- Найдите число элементов в каждой строке матрицы M , которые больше числа N (например, $N = 4$) (рис. 4.34).
- Определите, в каких строках матрицы M число m (например, $m = 7$) встречается ровно 2 раза? (рис. 4.35)
- Определите все пары столбцов матрицы M , сумма элементов которых больше K (например, $K = 75$) (рис. 4.36).

```

In [282]: M = rand(1:10, 6, 10)
Out[282]: 6×10 Matrix{Int64}:
 10  7  4  4  10  3  3  3  4  6
  6  9  3 10  6 10  1  5  1  1
 10  3  4  8  2  1  6  2  6  9
  7  5  1  1  9  6  5  9  8  5
  2  8  6  4  5  8  6  2  9  7
  3  6  9  3  6  4  7  3  3  4

In [288]: N = 4
rows = zeros(6)
for i in 1:6
    count = 0
    for j in 1:10
        if M[i, j] > N
            count += 1
        end
    end
    rows[i] = count
end
rows
Out[288]: 6-element Vector{Float64}:
 4.0
 6.0
 5.0
 8.0
 7.0
 4.0

```

Рисунок 4.34: Задание 9

```

In [291]: m = 8
          rows = zeros(6)
          for i in 1:6
              count = 0
              for j in 1:10
                  if M[i, j] == m
                      count += 1
                  end
              end
              if count == 2
                  rows[i] = 1
              end
          end
          rows

Out[291]: 6-element Vector{Float64}:
           0.0
           0.0
           0.0
           0.0
           1.0
           0.0

```

Рисунок 4.35: Задание 9

```

In [301]: k = 75
          for i in 1:10
              for j in (i+1):10
                  sum = 0
                  for r in 1:6
                      sum += M[r, i] + M[r, j]
                  end
                  if sum > k
                      println((i,j))
                  end
              end
          end

          (1, 2)
          (1, 5)
          (2, 5)

```

Рисунок 4.36: Задание 9

11. Вычислите 2 суммы (рис. 4.37).

```
In [303]: summa = 0
          for i in 1:20, j in 1:5
              summa += i^4 / (3+j)
          end
          summa
```

Out[303]: 639215.2833333334

```
In [304]: summa = 0
          for i in 1:20, j in 1:5
              summa += i^4 / (3+i*j)
          end
          summa
```

Out[304]: 89912.02146097136

Рисунок 4.37: Задание 9

5 Выводы

В ходе работы были изучены основы работы с функциями и циклами в я.п. Julia.

Список литературы